Non-linear least squares

gustavoc@msu.edu

2026-10-13

1) Ordinary Least Squares in the linear model

We have studied how to obtain ordinary least square (OLS) estimates for a linear model of the form

$$y = X\beta + \varepsilon$$

OLS estimates are obtained by minmizing the residual sum of squares with respect to the vector of regression coefficients (β).

$$RSS(y, X, \beta) = \Sigma_{i=1}^{n}(y_i - x_i'\beta)^2$$

Or, in matrix form

$$RSS(y, X, \beta) = (y - X\beta)'(y - X\beta) = y'y - 2\beta'X'y + \beta'X'X\beta$$

The gradient for with respect to β is

$$\frac{dRSS}{d\beta} = 2X'X\beta - 2X'y$$

Setting the gradient equal to zero we get

$$X'X\hat{\beta} = X'y$$

Assuming that $X'X$ is full rank, solving for β we get

$$\hat{\beta} = (X'X)^{-1}X'y$$

2) Non linear least squares

Suppose now that we want to fit a model of the form

$$y_i = f(x_i, \theta) + \varepsilon_i$$

Where $f(x_i, \theta)$ is a non-linear function of predictors (x_i) and model parameters (θ). We can estimates these parameters by least squares minmizing the RSS:

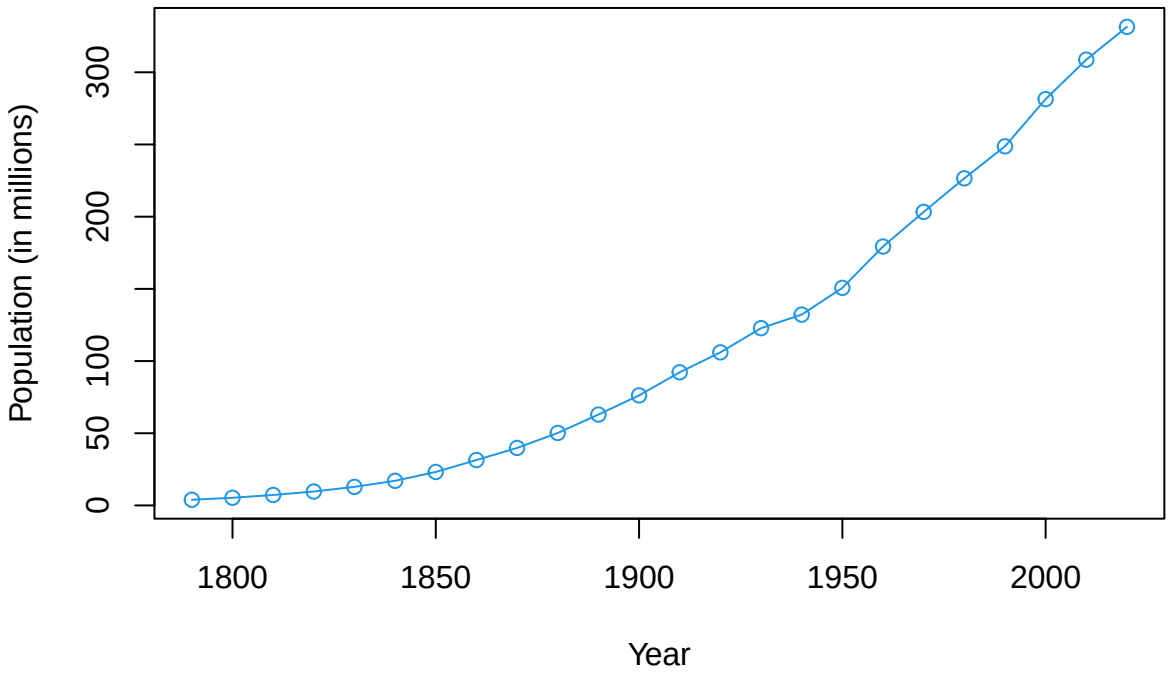
$$RSS(y, X, \theta) = \Sigma_{i=1}^{n}(y_i - f(x_i, \theta))^2$$

Typically, the least square estimates ($\hat{\theta}$) do not a closed form. However, we can always use numercial optimization methods to obtain estimates using an approach similar to the one we consider to derive Maximum likelihood estimates for the logistic regression models

3) The `nls()` function

The `nls()` R-function can be used to obtain least square estiamates for non-linear functions. The following example illustrates how to fit a logistic growth model to using US census data.

```
DATA=read.csv('https://raw.githubusercontent.com/gdlc/STAT_COMP/refs/heads/master/DATA/US_CENSUS.csv',
DATA$Population=DATA$Population/1e6 # population in millions
plot(Population~Year,data=DATA,type='o',col=4,ylab='Population (in millions)')
```



The logistic growth model has the following form

$$f(time) = \frac{\theta_1}{1 + exp(\theta_2 - x_i)/\theta_3}$$

Above:

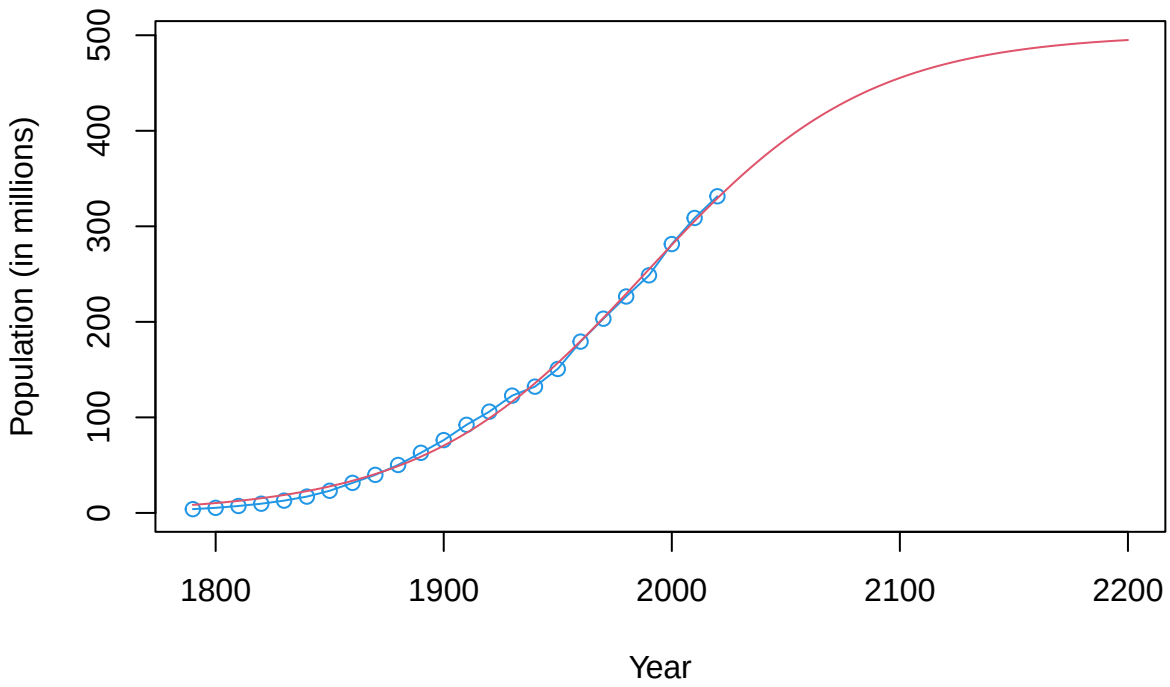
- θ_1 (named *Asym* in `SSlogis`) is an asymptote (the limit of $f()$ when $x \to \infty$.
- θ_2 (named *xmin* in `SSlogis`) is the inflection point of the curve, and
- θ_3 (names *scal* in `SSlogis`) is a scale parameter.

Fitting the curve using nls()

```
fm0=nls(Population~SSlogis(Year, Asym,xmid,scal),data=DATA)
summary(fm0)
```

```
##
## Formula: Population ~ SSlogis(Year, Asym, xmid, scal)
##
## Parameters:
##      Estimate Std. Error t value Pr(>|t|)
## Asym  501.458     29.055   17.26 6.99e-14 ***
## xmid 1988.332      5.973  332.89  < 2e-16 ***
## scal   48.767      1.828   26.67  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.019 on 21 degrees of freedom
##
## Number of iterations to convergence: 0
## Achieved convergence tolerance: 7.91e-06
```

```
bHat=summary(fm0)$coef[,1]
x=1790:2200
yHat=bHat[1]/(1+exp((bHat[2]-x)/bHat[3]))
plot(Population~Year,data=DATA,type='o',col=4,ylab='Population (in millions)',xlim=range(x),ylim=c(0,ma
lines(x=x,y=yHat,col=2)
```



The `nls()` function requires starting values (`start`); however, R has a few build functions (`SSlogis`, `SSasymp`, etc.) that generate their own starting values.

The following example fits the same model, in this case writing the function explicitly and providing starting values.

```r
# Using getIntial()
## start= getInitial(Population ~ SSlogis(Year, Asym, xmid, scal),
##                           data =DATA)

# Rough guess
start=c('Asym'=450,'xmid'=1960,'scal'=45)
fm1= nls(Population ~ Asym/(1 + exp((xmid - Year)/scal)),
                    data = DATA,
                    start = start)

summary(fm1)
```

```
##
## Formula: Population ~ Asym/(1 + exp((xmid - Year)/scal))
##
## Parameters:
##      Estimate Std. Error t value Pr(>|t|)
## Asym  501.458     29.054   17.26 6.99e-14 ***
## xmid 1988.332      5.973  332.89  < 2e-16 ***
## scal   48.767      1.828   26.67  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.019 on 21 degrees of freedom
##
## Number of iterations to convergence: 5
## Achieved convergence tolerance: 2.346e-06
```